Binary Search on Answer

Atonu Roy Chowdhury, atonuroy.chowdhury@bracu.ac.bd

June 30, 2026

Binary search is a very powerful searching tool when the search space is sorted. Usually, when we scan through an array to search for an element, in the worst case, we may need to scan through the whole array. Thus the searching takes $O(n)$ time. But if the array is sorted, we can speed up the process and search for an element in $O(\lg n)$ time.

§1 Binary search basics

Suppose you have a sorted array `arr`, and you wanna search for a value `x` in the array. The basic idea of binary search is that we halve the search space at every iteration. We compare `x` with the middle element of the array. If we get a match, then we're done. Otherwise, if the middle element is larger than `x`, then our new search space would be the left-half of the array. Or else, if the middle element is smaller than `x`, then our new search space would be the right-half of the array. We continue this until the search space becomes a singleton (i.e. a one-element array).

Below is a pseudo-code of binary search:

```
function BinarySearch(arr, x)
    left  = 0
    right = length(arr) - 1

    while left <= right do
        mid = (left + right) / 2

        if arr[mid] == x then
            return mid

        else if arr[mid] < x then
            left = mid + 1

        else
            right = mid - 1

    return NOT_FOUND
```

To analyze the complexity, we consider the value of `right` − `left`. It is initially n (the length of `arr`). At each step, it gets halved. And the loop runs until this quantity is 0. Therefore, the complexity of this algorithm is

$$O\left(\log_2 n\right) = O\left(\lg n\right).$$

Alternatively, we can find the complexity using Master theorem. To solve an n-size problem, we do some constant-time comparison and convert it into an $\frac{n}{2}$-size problem. Therefore, the recursive equation is

$$T\left(n\right) = T\left(\frac{n}{2}\right) + O(1).$$

This is of the form $T\left(n\right) = aT\left(\frac{n}{b}\right) + f(n)$, with $f\left(n\right) = O\left(n^{\log_b a}\right)$. Therefore, by Master theorem, the complexity is

$$T\left(n\right) = O\left(f(n)\lg n\right) = O\left(\lg n\right). \tag{1}$$

Using this tool, not only can we search for an element in an array, we can also solve other complicated problems that we're gonna see soon.

§2 Searching for the answer

Sometimes the search space may not be a simple array. The power of binary search is best understood when we search for the answer to a question in the space of all possible answers. To understand this, let's look at a problem first.

Problem 1. You are given an array containing n positive integers. Your task is to partition the array into at most k subarrays so that the maximum sum among all subarrays is as small as possible. Give the minimum possible largest subarray sum as output.

At first glance, the problem may look very confusing. So let's break the question down. Suppose $n = 5$, $k = 3$, and the array you're given is $[1, 2, 3, 4, 5]$. Your task is to partition the array into at most 3 subarrays (remember, subarrays are contiguous parts of an array). While doing a partition, you'll note down the sum of each subarrays in the partition. After you've performed a partition, you'll consider the maximum of the sums of subarrays. We have to minimize this maximum sum across all the partitions. [1]

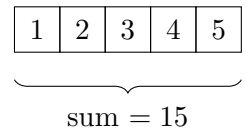
[1] Oftentimes, you'll see these type of questions are called min-max problem. This is because you are minimizing a maximum quantity. To write it more formally, a partition of an array into at most k-subarray entails

$$0 = i_0 < i_1 < i_2 < \cdots < i_m = n - 1, \qquad m \le k.$$

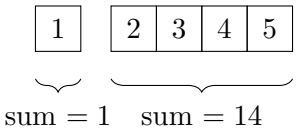
This defines m contiguous subarrays, where the j-th piece is $\left[a_{i_{j-1}+1}, \ldots, a_{i_j}\right]$. Then we have to find the following:

$$\min_{\substack{0=i_0<i_1<\cdots<i_m=n-1 \\ m\le k}} \max_{1\le j\le m} \sum_{t=i_{j-1}+1}^{i_j} a_t.$$

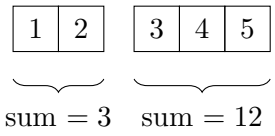
Let's see a visual example of the given array and $k = 3$. Here are all the partition of the array $[1, 2, 3, 4, 5]$ into at most 3 subarrays. First we have the partition into 1 subarray; then we have the partitios into 2 subarrays; finally, we have the partitions into 3 subarrays:



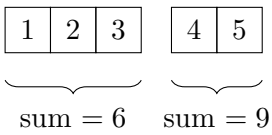
max subarray sum = 15



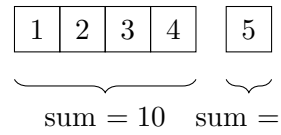
max subarray sum = 14



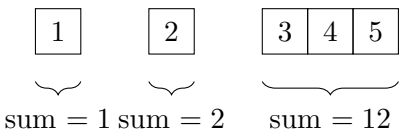
max subarray sum = 12



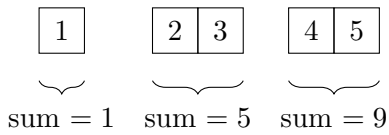
max subarray sum = 9



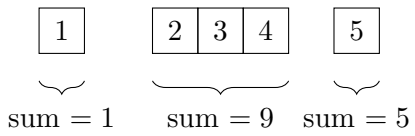
max subarray sum = 10



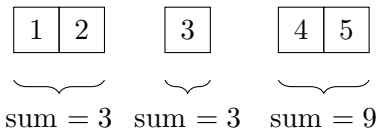
max subarray sum = 12



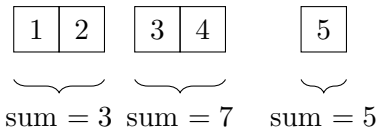
max subarray sum = 9



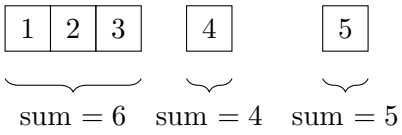
max subarray sum = 9



max subarray sum = 9



max subarray sum = 7



max subarray sum = 6

Now, notice that, across all these maximum subarray sums, the minimum value is 6. Therefore, the answer is 6.

Now, how do we actually solve this problem? In the aforementioned example, we used a brute-force approach. Can we really do it for large arrays?

Suppose $1 \leq i \leq k$. In how many ways we can partition an n-size array into i subarrays? Note that this is essentially placing $i-1$ separators in the $(n-1)$-many gaps between consecutive elements. Therefore,

$$\text{number of partitions into } i \text{ subarrays} = \binom{n-1}{i-1}.$$

To find the number of partitions into at most k subarrays, we need to add all these for $1 \leq i \leq k$.

$$\text{number of partitions into at most } k \text{ subarrays} = \sum_{i=1}^{k} \binom{n-1}{i-1}.$$

How big is this number actually?

- If k is a small constant, then this sum is dominated by term $\binom{n-1}{k-1}$, which is roughly $O\left(n^{k-1}\right)$.
- If $k = \frac{n}{2}$, then $\sum_{i=1}^{k} \binom{n-1}{i-1}$ is half the sum of the $(n-1)$-th row in Pascal's triangle. So this quantity is $\frac{1}{2} \cdot 2^{n-1} = O\left(2^n\right)$.
- If $k = n$, then $\sum_{i=1}^{k} \binom{n-1}{i-1}$ is the sum of the $(n-1)$-th row in Pascal's triangle. So this quantity is $2^{n-1} = O\left(2^n\right)$.

Therefore, the total number of partitions is exponential in the worst case. So we can't really solve the problem by brute-force. We have to think of a cleverer way.

Let's look at the problem from a different lens. Let's see how the answer changes based on the value of k. If $k = 1$, then there is only one partition, which is the whole array. In this case, the maximum subarray sum is the sum of all elements of the array.

$$\boxed{\text{If } k = 1, \text{ the answer is } \sum_{i=0}^{n-1} a_i.}$$

Let's now look at the other extreme. If $k = n$, then we can split the array into n-subarrays, i.e. all the subarrays can be singleton (i.e. array with only one element). In this case, the maximum subarray sum is the maximum element of the array.

$$\boxed{\text{If } k = n, \text{ the answer is } \max_{0 \le i \le n-1} a_i.}$$

(For notational simplicity, let's write $\texttt{lo} = \max_{0 \le i \le n-1} a_i$, and $\texttt{hi} = \sum_{i=0}^{n-1} a_i$.)

For other values of k, the answer will be somewhere in between `lo` and `hi`. So what we're gonna do is, we'll be searching for the answer between `lo` and `hi`. For this we'll use binary search.

But then we need to understand the following: in binary search, we halve the search space in each step. Here, initially, the search space is between `lo` and `hi`. Then how do we decide which half of the search space to choose?

For this purpose, we'll look at the problem from a different perspective. Instead of focusing on the answer itself, we'll look at k, i.e. the number of partitions. We'll decide which half of the search space to choose based on the following question:

> Given a fixed value `cap`, what's the minimum number of subarrays needed so that every subarray's sum $\le$ `cap`?

Actually, we'll answer whether this minimum number of subarrays is $\le k$ or not.

- If the answer is affirmative, i.e. if the minimum number of subarrays is $\le k$ and every subarray's sum $\le$ `cap`, we know that the actual answer is at most `cap`.
- If the answer is negative, i.e. if the minimum number of subarrays is $> k$ to ensure every subarray's sum stays below `cap`, then we know that the actual answer is larger than `cap`.

So our algorithm will be something like this:

1. We'll do binary search between `lo` and `hi`.
2. Take the midpoint of them. Call it `mid`.
3. Check whether it's possible to have a partition into at most k subarrays so that every subarray's sum is $\le$ `mid`.
4. If the answer is affirmative, choose the left-half of the search space. Otherwise choose the right-half of the search space.
5. Repeat until `lo` = `hi`. Then return the value.

We haven't yet found a way to check whether the minimum number of subarrays is $\le k$ to ensure that every subarray's sum $\le$ `cap`. Suppose that is written in a boolean function called `possible`. Then our pseudocode looks like:

```
function solve(arr, n, k)
    lo  = max(arr)
    hi = sum(arr)

    while lo < hi do
        mid = (lo + hi) / 2

        if possible(mid) then
            hi = mid

        else
            lo = mid + 1

    return lo
```

Now, let's see how we can write the function `possible`. We'll scan the array from first to last. In doing so, we'll take as many numbers as possible in a subarray until the taken numbers' sum exceeds `cap`. If the sum exceeds `cap`, we'll start a new subarray. After all these, we'll check whether the number of subarrays is $\leq k$ or not.

In the following pseudocode, `pieces` is the number of subarrays, `currSum` is the sum of currently taken numbers.

```
function possible(cap)
    pieces = 1, currSum = 0

    for x in arr do
        currSum += x

        if currSum > cap then
            pieces++ , currSum = x

            if pieces > k then
                return False

    return True
```

If `currSum > cap`, we start a new subarray with `x`, so `currSum = x`. Also, we increment `pieces`.

Now, what's the time complexity of this whole program? Binary search runs in logarithmic scale of the search space. Here, the upper limit of the search space is $\sum_{i=0}^{n-1} a_i$. Therefore, the search takes $\lg\left(\sum_{i=0}^{n-1} a_i\right)$-many iterations. At each iteration, we run the function `possible` once, which takes $O(n)$ time. Therefore, the overall complexity is

$$O\left(n \cdot \lg\left(\sum_{i=0}^{n-1} a_i\right)\right).$$

§3 Some more practice problems

Problem 1. You are given n ropes with positive integer lengths. You may cut the ropes into smaller pieces (each with positive integer length).

Your task is to obtain at least k pieces such that every piece has the same positive integer length. Any leftover part after cutting may be discarded.

Find the maximum possible length of each piece. If it is not possible to obtain at least k pieces of any positive integer length, print -1.

Problem 2. (From [3]) You are given an array `arr` consisting of n integers which denote the position of a stall. You are also given an integer k which denotes the number of aggressive cows.

Your task is to assign stalls to k cows such that the minimum distance between any two of them is the maximum possible. Return the maximum possible minimum distance.

Problem 3. Koko loves to eat bananas. There are n piles of bananas, the i-th pile has `piles[i]` bananas. The guards have gone and will come back in h hours.

Koko can decide her bananas-per-hour eating speed of k. Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return. Return the minimum integer k such that she can eat all the bananas within h hours.

Problem 4. (From [4]) In an $n \times m$ matrix, where the (i, j)-th entry is $i \cdot j$, find the k-th smallest element.

You can check [2] for more problems.

References

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, Cambridge, MA, 2009.

[2] Zhijun Liao, "[Python] Powerful Ultimate Binary Search Template. Solved Many Problems," *LeetCode Discuss*, Aug. 11, 2020 (updated Aug. 2, 2022). Available: https://leetcode.com/discuss/post/786126. Accessed: Jun. 30, 2026.

[3] Sphere Online Judge (SPOJ), "AGGRCOW – Aggressive Cows." Available: https://www.spoj.com/problems/AGGRCOW/. Accessed: Jun. 30, 2026.

[4] Codeforces, "448D – Multiplication Table," Codeforces Round 256 (Div. 2). Available: https://codeforces.com/problemset/problem/448/D. Accessed: Jun. 30, 2026.